M01 — Foundations of Algorithm Design

Sources: CLRS Ch. 1 (The Role of Algorithms in Computing), CLRS Ch. 2 (Getting Started) · Skiena Ch. 1 (Introduction to Algorithm Design)

Big Idea

An algorithm is a *procedure that provably transforms every legal input into the required output in finite time*. The word that carries the weight is **every**. A procedure that works on the instances you happened to try is a **heuristic**, not an algorithm — and the gap between the two is where almost all real engineering mistakes live. This module builds the three habits that everything later depends on: (1) **specify the problem precisely** — allowed inputs and required output properties — before writing a line; (2) **attack your own algorithm with counterexamples** before defending it; (3) **prove what survives**, usually by induction, expressed for loops as a **loop invariant**. Alongside correctness sits the second axis, efficiency, introduced here through the insertion sort vs. merge sort comparison: the *order of growth* of running time dominates constant factors and hardware speed once inputs are large. Months from now, the thing to remember is the discipline: *specify → attempt → break → prove → analyze*, and that "it's obvious" is never a proof.

What You Should Be Able To Do After This Chapter

- Write a problem specification as an explicit **Input:** / **Output:** pair, and spot when a specification is too broad, ill-defined, or a compound goal.
- Take a plausible-looking greedy or incremental heuristic and **construct a small counterexample** using a repeatable method (think small, think exhaustively, go for a tie, seek extremes).
- State a **loop invariant** for an iterative algorithm and discharge the three obligations: initialization, maintenance, termination.
- Prove a recursive algorithm correct by induction, including strengthening the hypothesis when the recursion drops by more than one.
- Model a vague real-world task in terms of **permutations, subsets, trees, graphs, points, polygons, strings** — and see the recursive decomposition of each.

- Set up and read a divide-and-conquer recurrence such as `T(n) = 2T(n/2) + θ(n)` and explain via a recursion tree why it solves to `θ(n log n)`.
- Explain why an asymptotically better algorithm on slow hardware beats an asymptotically worse one on fast hardware, and estimate the crossover.
- Implement insertion sort and merge sort correctly in C++ from memory, including the merge boundary conditions.

---

1. What an algorithm is, and what a problem is

Unified Understanding

Problem vs **instance** vs **algorithm** — three distinct things that beginners conflate.

- A **problem** is a specification: a description of the set of legal *inputs* and the *properties* the output must satisfy. [Skiena §1.3.1, p.11]
- An **instance** is one particular input satisfying those constraints. [CLRS §1.1, p.6]
- An **algorithm** is a specific computational procedure achieving that input/output relationship **for all instances**. [CLRS §1.1, p.5]

The canonical specification format, used identically by both books:

```
Problem: Sorting
Input:   A sequence of n numbers ⟨a₁, a₂, …, aₙ⟩.
Output:  A permutation ⟨a′₁, a′₂, …, a′ₙ⟩ of the input such that a′₁
≤ a′₂ ≤ … ≤ a′ₙ.
```

Correctness (CLRS's definition). An algorithm is *correct* if for **every** problem instance it **halts** in finite time and outputs a correct solution. An incorrect algorithm may fail to halt on some inputs, or halt with a wrong answer. [CLRS §1.1, p.6]

The one caveat worth remembering: CLRS explicitly notes that incorrect algorithms are *sometimes useful if you can control the error rate* — the Miller–Rabin primality test in CLRS Ch. 31 is the example they promise. So "incorrect" is not automatically "useless"; "incorrect and uncharacterized" is.

Keys and satellite data. When we sort, we sort *records*: a **key** plus **satellite data**. Analysis focuses on keys, but implementations move whole records — which is why the cost of a swap is not always O(1) in practice. [CLRS §2.1, p.17]

Three desirable properties (Skiena's framing)

Skiena states the design tension that CLRS leaves implicit:

> *We seek algorithms that are **correct** and **efficient**, while being **easy to implement**. These goals may not be simultaneously achievable. [Skiena §1, p.3]*

And the industrial reality check:

> *In industrial settings, any program that seems to give good enough answers without slowing the application down is often acceptable, regardless of whether a better algorithm exists. The issue of finding the best possible answer or achieving maximum efficiency usually arises in industry only after serious performance or legal troubles.*

Skiena emphasis: the third axis (implementability) and the pragmatic threshold. **CLRS emphasis:** correctness is binary and non-negotiable; efficiency is then optimized.

Recognition pattern

If a task description contains an undefined superlative — "best route", "most relevant", "optimal schedule" — the specification is not yet a problem. Pin down the objective before designing.

---

2. Algorithms vs. heuristics — the counterexample discipline

This is Skiena's Chapter 1 and it has no counterpart in CLRS. It is also the single most interview-relevant part of this module.

Problem

You propose a reasonable rule. Is it actually correct?

Case study 1: Robot tour optimization (TSP)

```
Problem: Robot Tour Optimization
Input:   A set S of n points in the plane.
Output:  The shortest cycle tour that visits each point in S.
```

[Skiena §1.1, p.5]

Attempt 1 — Nearest neighbour.

```
NearestNeighbor(P)
    Pick and visit an initial point p₀ from P
    p = p₀;  i = 0
    While there are still unvisited points
        i = i + 1
        Select pᵢ to be the closest unvisited point to pᵢ₋₁
        Visit pᵢ
    Return to p₀ from pₙ₋₁
```

→ **C++ implementation:** A1 NearestNeighbor

Simple, fast, intuitive — and **wrong**. Counterexample: points on a line at positions `-21, -5, -1, 0, 1, 3, 11`. Starting at `0`, the rule hops left–right–left–right across the origin. The optimal tour sweeps from the leftmost point rightward and returns. [Skiena Fig. 1.3, p.6]

The natural patch ("start from the leftmost point") dies immediately: rotate the instance 90°, and now all points are equally leftmost. **No choice of starting rule saves nearest-neighbour.**

Attempt 2 — Closest pair. Repeatedly join the closest pair of endpoints from distinct chains, avoiding premature cycles; connect the final two endpoints.

```
ClosestPair(P)
    Let n = |P|
    For i = 1 to n − 1
        d = ∞
        For each pair of endpoints (s, t) from distinct vertex
chains
            if dist(s, t) ≤ d then sₘ = s, tₘ = t, d = dist(s, t)
        Connect (sₘ, tₘ) by an edge
    Connect the two remaining endpoints by an edge
```

→ **C++ implementation:** A2 ClosestPair

This fixes the previous counterexample — and then dies on a new one: two rows of points, rows separated by `1 − ε`, neighbours within a row separated by `1 + ε`. The closest pairs stretch *across* the gap rather than along the rows, and the resulting tour is over 20% longer than optimal as `ε → 0`. [Skiena Fig. 1.4, p.8]

Attempt 3 — Exhaustive. Enumerate all `n!` orderings, keep the best. Correct, and useless: `20! ≈ 2.4 × 10¹⁸`.

Take-home lesson (Skiena, verbatim in spirit): there is a fundamental difference between **algorithms**, which always produce a correct result, and **heuristics**, which may usually do a good job but provide no guarantee.

Case study 2: Movie scheduling — where a greedy rule *does* work

```
Problem: Movie Scheduling
Input:   A set I of n intervals on the line.
Output:  The largest subset of mutually non-overlapping intervals
from I.
```

[Skiena §1.2, p.9]

HEURISTIC	COUNTEREXAMPLE	DAMAGE
Earliest start first	One very long job (*War and Peace*) starts first and blocks everything	unbounded
Shortest job first	A short job wedged between two others blocks both	up to ½ of optimal
Fewest conflicts first	Chain of 7 intervals, piled at the ends	3 instead of 4
Exhaustive over `2ⁿ` subsets	correct	`2¹⁰⁰` — hopeless
Earliest completion first	**none — provably optimal**	—

Why earliest-completion works (the exchange argument in embryo): let `x` be the interval whose *right* endpoint is leftmost. Every other interval that starts before `x` ends must overlap `x`, so at most one interval from that whole group can be chosen. Of those candidates, `x` is the one that frees the timeline soonest, so it never blocks anything a competitor wouldn't also block. Therefore you can never lose by picking `x`. Recurse on what remains. *(Formalized as an exchange argument in M12 — Greedy; CLRS's activity-selection problem, §15.1, is exactly this problem.)*

```
OptimalScheduling(I)
    While (I ≠ ∅)
        Accept the job j from I with the earliest completion date
        Delete j, and any interval intersecting j, from I
```

→ **C++ implementation:** A3 OptimalScheduling

Skiena's counterexample-hunting toolkit

This is a *method*, not a knack. Memorize it — it is what senior interviewers are probing when they say "are you sure?"

TECHNIQUE	WHAT TO DO
Think small	Real counterexamples are tiny. The TSP one had ⩽ 6 points; the scheduling ones had 3 intervals. Amateurs draw a huge messy instance and stare; pros check several small ones.
Think exhaustively	For the first non-trivial `n`, enumerate *all* structurally distinct instances. Two intervals on a line have exactly three arrangements: disjoint, overlapping, nested. Build all 3-interval cases by extending those.
Hunt for the weakness	If the rule is "always take the biggest / closest / shortest", ask what that choice forecloses.
Go for a tie	Make everything the same size. Now the heuristic has no basis for its decision and is free to pick badly.
Seek extremes	Mix huge and tiny, near and far, few and many. Two tight clusters separated by distance `d` make the optimal TSP tour ≈ `2d` regardless of cluster contents.

A good counterexample has two properties [Skiena §1.3.3, p.13]:

- **Verifiability** — you can compute what the algorithm returns *and* exhibit a better answer.
- **Simplicity** — every unnecessary detail stripped, so you can hold it in your head.

Recognition pattern

Any time you are about to say "just always pick the X-est", spend 60 seconds on: *tie, extreme, small, exhaustive*. This alone will catch the majority of wrong greedy answers in interviews.

---

3. Specifying problems well

Three failure modes [Skiena §1.3.1, p.11]

1. Input class too broad. Allow movie projects to have *gaps* (film in September and November, hiatus in October) and each project becomes a *set* of intervals. Earliest-completion breaks, and in fact no efficient algorithm exists — the generalized version is NP-hard.

> ***Take-home lesson:*** *An important and honorable technique in algorithm design is to* ***narrow the set of allowable instances*** *until there is a correct and efficient algorithm. Restrict a graph problem from general graphs down to trees; restrict a geometric problem from 2-D to 1-D.*

This is not cheating. It is the single most productive move in applied algorithm design, and it is exactly what an interviewer is inviting when they ask "what if the input were sorted / a tree / bounded in value?"

2. Ill-defined question. "Best route between two places" is not a problem until you say *shortest in distance*, *fastest in time*, or *fewest turns*. These are three different answers.

3. Compound goals. "Find the shortest route from `a` to `b` that doesn't use more than twice as many turns as necessary" is perfectly well defined and horrible to reason about. **Pick one criterion.** Multi-objective versions are usually much harder — often NP-hard where each single objective was polynomial.

Recognition pattern

If the requirement has an "and also" or a "but not too much" in it, you are looking at a constrained-optimization problem, not a shortest-path problem. Expect DP, Lagrangian relaxation, or NP-hardness.

---

4. Proving correctness: induction and loop invariants

Two notations for one idea. Skiena presents induction; CLRS presents its loop form.

> *Recursion* ***is*** *mathematical induction in action. In both, we have general and boundary conditions, with the general condition breaking the problem into smaller pieces, and the initial/boundary condition terminating it. [Skiena §1.4, p.15]*

The loop invariant method (CLRS §2.1, p.20)

A **loop invariant** is a property of the program state that is true at the top of every iteration. To use one you must show three things:

OBLIGATION	STATEMENT	INDUCTION ANALOGUE
Initialization	It is true prior to the first iteration.	base case
Maintenance	If true before an iteration, it remains true before the next.	inductive step
Termination	The loop terminates, and the invariant *plus the reason the loop exited* gives the property you wanted.	— (this is the payoff)

The third is where the proof actually happens, and it is the one people skip. The pattern is always: *substitute the terminating value of the loop variable into the invariant and read off the theorem.*

Crucial CLRS convention that makes termination arguments clean: the loop counter **retains its value after the loop exits**, and for a `for` loop that value is the first one that exceeded the bound. So after `for i = 2 to n`, we have `i = n + 1`. [CLRS §2.1, p.22]

Induction pitfalls [Skiena §1.4, p.16]

Boundary errors. The insertion-sort argument says "there is a unique spot between the largest element ⩽ x and the smallest element > x" — which quietly assumes both exist. Inserting a new minimum or maximum needs separate care.

Cavalier extension claims. Adding *one* element can change the *entire* optimal solution. In movie scheduling, the optimal schedule after inserting one new interval may contain **none** of the intervals from a previous optimal solution [Skiena Fig. 1.8, p.16]. "The optimal solution for `n` is the optimal solution for `n−1` plus one more item" is the most common false inductive step in interviews, and it is exactly what optimal substructure (M11) has to be *proved*, not assumed.

Strengthening the hypothesis. If the recursion drops from `n` to `n/2`, an inductive hypothesis about `n − 1` is useless. Strengthen it to *"holds for all $y \leqslant n - 1$"* (strong induction). Skiena's worked example:

```
Increment(y)
    if (y = 0) return 1
    if (y mod 2) = 1 then return 2 · Increment(⌊y/2⌋)
    else return y + 1
```

→ **C++ implementation:** A4 Increment

For odd $y = 2m + 1$:
$2 \cdot \text{Increment}(\lfloor (2m+1)/2 \rfloor) = 2 \cdot \text{Increment}(m) = 2(m + 1) = 2m + 2 = y + 1$ ✓

This costs nothing in principle but is *necessary* for the proof to close. [Skiena §1.4, p.17]

Proof by contradiction [Skiena §1.6, p.21]

The scheme:

1. Assume the statement you want to prove is **false**.
2. Derive logical consequences.
3. Show one consequence is **demonstrably, ridiculously false**.

Euclid's infinitude of primes is the model: assume primes are exactly $p_1 \ldots p_m$, form $N = \prod p_i$, and observe $N + 1$ is divisible by none of them.

> *For a contradiction argument to be convincing, the final consequence must be **clearly, ridiculously false**. Muddy outcomes are not convincing.*

Where you will use it: minimum spanning tree cut/cycle properties (M14), greedy exchange arguments (M12), and sorting lower bounds (M05).

Proof-technique picker

ALGORITHM SHAPE	PROOF TECHNIQUE
Iterative, builds solution incrementally	**loop invariant**
Recursive, `n → n−1`	**induction**
Recursive, `n → n/b`	**strong induction**
Greedy	**exchange argument** + greedy-choice property
Optimality of a structure (MST, shortest path)	**contradiction** / cut property
Lower bound	**adversary / information-theoretic counting**
Divide & conquer running time	**recurrence** (M03)

---

5. Modeling: reducing your application to known structures

> ***Modeling is the art of formulating your application in terms of precisely described, well-understood problems.*** *Proper modeling is the key to applying algorithmic design techniques to real-world problems. Indeed, proper modeling can eliminate the need to design or even implement algorithms, by relating your application to what has been done before. [Skiena §1.5, p.17]*

You cannot look up "widget optimization". You *can* look up "minimum vertex cover on an interval graph". The whole point of modeling is to make your problem findable.

The seven fundamental combinatorial objects

OBJECT	WHAT IT REPRESENTS	TRIGGER WORDS IN THE PROBLEM STATEMENT
Permutations	arrangements / orderings	"arrangement", "tour", "ordering", "sequence"
Subsets	selections (order irrelevant)	"cluster", "collection", "committee", "group", "packaging", "selection"
Trees	hierarchical relationships	"hierarchy", "dominance relationship", "ancestor/descendant", "taxonomy"
Graphs	relationships between arbitrary pairs	"network", "circuit", "web", "relationship"
Points	locations in geometric space	"sites", "positions", "data records", "locations"
Polygons	regions in geometric space	"shapes", "regions", "configurations", "boundaries"
Strings	sequences of characters / patterns	"text", "characters", "patterns", "labels"

This table is worth memorizing. In an interview, the mapping from the story to the object *is* the first half of the answer.

Every one of them is recursive [Skiena §1.5.2, p.19]

OBJECT	DECOMPOSITION	BASE CASE
Permutation	delete the first element (renumber to keep it a permutation of `1..n-1`)	`{}`
Subset	every subset of `{1..n}` contains a subset of `{1..n-1}` (drop `n` if present)	`{}`
Tree	delete the root → a forest of smaller trees; delete a leaf → a slightly smaller tree	single vertex
Graph	delete a vertex → smaller graph; cut left/right → two graphs + broken edges	single vertex
Points	separate with a line → two smaller clouds	single point
Polygon	insert an internal chord between non-adjacent vertices → two polygons	triangle
String	delete the first character → shorter string	empty string

Why this matters: the "cut left/right → two graphs + broken edges" decomposition is exactly divide-and-conquer on graphs; the "delete a vertex" one is exactly the DP/backtracking recursion. Every design technique in this book is a way of exploiting one of these decompositions.

Modeling caveats

Skiena is careful here, and it is worth quoting the balance:

> *The act of modeling reduces your application to one of a small number of existing problems and structures. Such a process is inherently constraining, and certain details might not fit easily into the given target problem. Also, certain problems can be modeled in several different ways, some much better than others. … Be alert for how the details of your applications differ from a candidate model, but* ***don't be too quick to say that your problem is unique and special****.*

War Story: Psychic Modeling — the cost of modeling wrong

Skiena's lottery-covering story [§1.8, p.22]. A client wants the smallest set of lottery tickets guaranteeing a prize, given a psychic's promise that $\geqslant$ `j` of `n` candidate numbers will be drawn. Skiena correctly identified it as a **set cover** instance (NP-complete), correctly chose the components (subset ranking/unranking, a bit-vector for the covered

set, simulated annealing for the search) — and then **modeled the covering criterion wrong**, requiring every winning `l`-subset to appear explicitly in some purchased ticket. The client produced a 5-ticket solution against Skiena's "optimal" 28.

The framework survived; only the definition of "covered" was wrong.

> *The moral:* ***make sure that you model your problem correctly before trying to solve it.*** *… Our misinterpretation would have become obvious had we worked out a small example by hand and bounced it off our sponsor before beginning work.*

Engineering translation: hand-verify your model on a 5-element instance with the stakeholder before you write code. This is the same "think small" discipline as counterexample hunting, applied to specification rather than to algorithms.

---

6. The RAM model of computation

Unified Understanding

To compare algorithms without implementing them, we need a machine model. Both books use the **RAM** (Random Access Machine).

The rules [Skiena §2.1, p.31; CLRS §2.2, p.26]:

- Each **simple operation** (`+`, `*`, `-`, `=`, `if`, `call`) takes **exactly one time step**.
- **Loops and subroutines are not simple** — they are compositions of many single-step operations. `sort` cannot be one step, since sorting 10^6 items obviously takes longer than sorting 10.
- Each **memory access takes one time step**, including array indexing, and memory is unlimited.
- Instructions execute **sequentially**, no concurrency.

CLRS's extra precision, worth knowing:

- The instruction set is *what real computers have*: arithmetic (add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), control (branch, call, return).
- **Word size is bounded.** For input size `n`, integers are represented in `c·log₂ n` bits for some constant `c ≥ 1`. `c ≥ 1` so a word can hold `n` (needed to index the input); `c` constant so we cannot smuggle unbounded data into one word and

operate on it in O(1).

- **Grey areas.** Is exponentiation constant time? Generally *no* — computing x^n takes time logarithmic in n. But 2^n via a left shift is one instruction *when the result fits in a word*. CLRS's rule: treat computing 2^n and multiplying by 2^n as O(1) **when the result fits in a word**, and otherwise don't.

Assumptions and what they cost you

The RAM model does not model the memory hierarchy. No caches, no virtual memory. [CLRS §2.2, p.27; Skiena §2.1, p.31]

CLRS's honest defense: models that include the hierarchy are substantially more complex and harder to work with, and **RAM-model analyses are usually excellent predictors of real performance**. Where it matters, CLRS handles it in §11.5 (hash tables) and a handful of problems.

Outside / Engineering Context

Neither book develops this, but for competitive programming and production work the RAM model's blind spot is worth naming: on modern hardware a cache miss costs ~100× an L1 hit, so two algorithms with identical `Θ(n)` bounds can differ by an order of magnitude. Concretely — `std::vector` traversal beats `std::list` traversal badly; a flat array-of-structs beats a pointer-chasing tree for small `n`; and `unordered_map` with its bucket-of-nodes layout loses to a sorted `vector` + binary search for `n` in the low thousands. Use the RAM model to *choose the algorithm*, then measure.

---

7. Best, worst, and average case

Formal understanding

Define, over the set of all instances of size n:

CASE	DEFINITION	SYMBOL
Worst case	maximum number of steps over any instance of size n	$T_{worst}(n)$
Best case	minimum number of steps over any instance of size n	$T_{best}(n)$
Average case	expected number of steps over a *stated* input distribution	$T_{avg}(n)$

Note the three are functions of `n`, obtained by taking max/min/mean over the (finite) set of size-`n` instances.

Why CLRS defaults to worst case [CLRS §2.2, p.31]

1. It is an **upper bound guarantee**: the algorithm never takes longer. Essential for real-time systems with deadlines.
2. **The worst case often occurs.** Searching a database for absent information hits the worst case every time, and absent-key lookups are common.
3. **The average case is often roughly as bad.** Insertion sort on random input compares against about half the sorted prefix, so `tᵢ ≈ i/2` — still quadratic.

Why average case is hard

> *The scope of average-case analysis is limited, because it may not be apparent what constitutes an "average" input for a particular problem. Often, we'll assume that all inputs of a given size are equally likely. In practice, this assumption may be violated. [CLRS §2.2, p.32]*

The escape hatch, and it is the important idea: use a **randomized algorithm**, which makes random choices *itself*. Then the expectation is over the algorithm's coin flips rather than over an assumed input distribution — so there is no bad input, only unlucky runs. This is the whole justification for randomized quicksort (M05) and is developed in M04.

Common mistakes

- Confusing "average case" with "worst case with a smaller constant". They are different quantifications.
- Reporting best case as if it were meaningful. Any sorting algorithm can be given an `O(n)` best case by prepending a sortedness check — CLRS makes this an exercise (2.2-4) precisely to show the metric is gameable.
- Assuming uniform input distribution without saying so.

---

8. Algorithm: Insertion Sort

Problem

Sort `A[1..n]` into monotonically increasing order, in place.

Core intuition

Sorting a hand of playing cards. The left hand always holds a sorted prefix. Take the next card from the table and slide it left past every larger card until it lands. [CLRS §2.1, p.18]

Mental model

Sorted prefix grows by one each iteration; the new element bubbles left into place.

Algorithm

For `i = 2..n`: save `key = A[i]`; shift every element of `A[1..i−1]` greater than `key` one position right; drop `key` into the hole.

Pseudocode (CLRS, 1-indexed)

```
INSERTION-SORT(A, n)
1  for i = 2 to n
2      key = A[i]
3      // Insert A[i] into the sorted subarray A[1..i−1]
4      j = i − 1
5      while j > 0 and A[j] > key
6          A[j + 1] = A[j]
7          j = j − 1
8      A[j + 1] = key
```

→ **C++ implementation:** A5 INSERTION-SORT

Correctness intuition

At the top of each `for` iteration the prefix left of `i` is a sorted rearrangement of the elements that started there. The inner loop makes room for `A[i]` without losing anything, because it *copies* rightward into a slot whose value is already saved in `key`.

Proof skeleton — loop invariant

> ***Invariant:*** *At the start of each iteration of the `for` loop of lines 1–8, the subarray `A[1..i−1]` consists of the elements originally in `A[1..i−1]`, but in sorted order.*

- **Initialization.** `i = 2`, so `A[1..1]` is one element: trivially sorted, and trivially the original element.
- **Maintenance.** The body shifts `A[i−1], A[i−2], …` right until it finds the right position for `A[i]` (lines 4–7), then inserts it (line 8). So `A[1..i]` now consists of the original elements of `A[1..i]` in sorted order. Incrementing `i` restores the invariant. *(A fully formal treatment would state a nested invariant for the `while` loop — CLRS explicitly declines to, and so do we.)*
- **Termination.** `i` starts at 2, increments by 1, and the loop exits when `i = n + 1`. Substituting `i = n + 1` into the invariant: `A[1..n]` consists of the original elements in sorted order. ∎

This is the template. Every loop-invariant proof you will ever write is this shape.

Complexity — derived, not asserted

Let t_i = number of times the `while` test on line 5 executes for that `i`. (A loop test runs one more time than the body.) Summing cost × times:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1) + c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1)$$

Best case — already sorted. `key ≥` everything in `A[1..i−1]`, so the `while` exits on its first test: $t_i = 1$.

$$T(n) = (c_1+c_2+c_4+c_5+c_8)\cdot n - (c_2+c_4+c_5+c_8) \;=\; an + b \;\to\; \Theta(n)$$

Worst case — reverse sorted. Every comparison fails, `while` runs until `j = 0`: $t_i = i$.

Using $\sum_{i=2}^{n} i = n(n+1)/2 - 1$ and $\sum_{i=2}^{n} (i-1) = n(n-1)/2$:

$$T(n) = (c_5/2 + c_6/2 + c_7/2)\cdot n^2 + (\ldots)\cdot n - (\ldots) \;=\; an^2 + bn + c \;\to\; \Theta(n^2)$$

Average case. On random input, `A[i]` is compared against roughly half of `A[1..i−1]`, so $t_i \approx i/2$. Halving a quadratic leaves a quadratic: **$\Theta(n^2)$**.

	TIME	SPACE
Best	$\Theta(n)$	$\Theta(1)$ auxiliary
Average	$\Theta(n^2)$	$\Theta(1)$
Worst	$\Theta(n^2)$	$\Theta(1)$

Where the complexity comes from: the dominant work is line 6, the rightward shift. The number of shifts equals **the number of inversions** in the input — pairs `(i, j)` with `i < j` and `A[i] > A[j]` [CLRS Problem 2-4]. So insertion sort is `θ(n + inv(A))`, which is the sharpest statement of its behaviour and immediately explains both the `θ(n)` best case (`inv = 0`) and the `θ(n²)` worst case (`inv = n(n−1)/2`, reverse-sorted).

C++ Implementation

```cpp
#include <vector>

// Sorts v in place, ascending. Stable. O(n + inversions) time,
O(1) extra space.
template <typename T>
void insertionSort(vector<T>& v) {
    const int n = static_cast<int>(v.size());
    for (int i = 1; i < n; ++i) {          // 0-indexed: prefix [0,
i) is sorted
        T key = move(v[i]);
        int j = i - 1;
        // Strict '>' keeps the sort stable: equal elements never
cross.
        while (j >= 0 && key < v[j]) {
            v[j + 1] = move(v[j]);
            --j;
        }
        v[j + 1] = move(key);
    }
}
```

Implementation notes

- **`j >= 0` must be tested before `v[j]`** — C++ `&&` short-circuits, which is exactly why this is safe. CLRS calls out short-circuiting as a deliberate pseudocode convention for the same reason [CLRS §2.1, p.24].
- **Stability** comes from `key < v[j]` (strict). Using `<=` would swap equal elements

and destroy stability.

- `std::move` matters when `T` is expensive to copy (`std::string`, vectors of vectors). For `int` it compiles away.
- Skiena's C version uses repeated `swap`; the shift-and-place version above does one write per displaced element instead of three, and is meaningfully faster.

Common bugs

- Starting the outer loop at `i = 0` (nothing to insert into) or at `i = 1` in 1-indexed code (skips an element).
- Writing `v[j] = key` instead of `v[j + 1] = key` after the loop. Off by one, silently corrupts.
- Testing `v[j] > key` **before** `j >= 0`.
- Using `<=` in the comparison and then relying on stability elsewhere (e.g. in a radix sort pass — see M05).

Recognition pattern

Reach for insertion sort when: `n` is small (≲ 30–50), or the array is **nearly sorted** (few inversions), or you need a **stable, in-place, online** sort — insertion sort can absorb elements as they arrive. It is the standard base case inside introsort/merge sort for exactly these reasons [CLRS Problem 2-1].

Alternatives

ALTERNATIVE	WHEN PREFERABLE
Merge sort	Large `n`, need `Θ(n log n)` worst case, can afford `Θ(n)` space
Quicksort	Large `n`, in-place, average case matters more than worst
Heapsort	Large `n`, need `Θ(n log n)` worst case *and* `O(1)` space
Counting/radix sort	Keys are small integers (M05)
Insertion sort	small `n`, nearly-sorted, stability + in-place + online

---

9. The divide-and-conquer method

Core idea [CLRS §2.3.1, p.34]

If the problem is small enough — the **base case** — solve it directly. Otherwise, the **recursive case** performs three steps:

- **Divide** the problem into one or more subproblems that are smaller instances of the same problem.
- **Conquer** the subproblems by solving them recursively.
- **Combine** the subproblem solutions into a solution to the original.

Contrast with insertion sort, which uses the **incremental method**: having sorted `A[1..i−1]`, insert `A[i]`.

Why D&C is worth the trouble: analyzing it is often straightforward, because the three steps translate mechanically into a recurrence. Full treatment in M03.

---

10. Algorithm: Merge Sort

Problem

Sort `A[p..r]` into increasing order.

Core intuition

Two sorted piles of cards face up, smallest on top. Repeatedly take the smaller of the two exposed cards. When one pile empties, flip the rest of the other onto the output. [CLRS §2.3, p.35]

Mental model

Split in half, sort each half, zip them together. All the work is in the zip.

Algorithm

- **Divide:** `q = ⌊(p + r)/2⌋`, splitting `A[p..r]` into `A[p..q]` (`⌈n/2⌉` elements) and `A[q+1..r]` (`⌊n/2⌋` elements).
- **Conquer:** recursively merge-sort each half.

- **Combine:** `MERGE(A, p, q, r)`.
- **Base case:** `p ≥ r` — zero or one element, already sorted.

Pseudocode

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint
4  MERGE-SORT(A, p, q)
5  MERGE-SORT(A, q + 1, r)
6  MERGE(A, p, q, r)

MERGE(A, p, q, r)
1  n_L = q − p + 1;  n_R = r − q
2  let L[0..n_L−1] and R[0..n_R−1] be new arrays
3  copy A[p..q] into L;  copy A[q+1..r] into R
4  i = 0;  j = 0;  k = p
5  while i < n_L and j < n_R
6      if L[i] ≤ R[j]
7          A[k] = L[i];  i = i + 1
8      else
9          A[k] = R[j];  j = j + 1
10     k = k + 1
11 while i < n_L:  A[k] = L[i];  i = i + 1;  k = k + 1
12 while j < n_R:  A[k] = R[j];  j = j + 1;  k = k + 1
```

→ **C++ implementation:** A6 MERGE-SORT and MERGE

Why it works

The merge maintains: *`A[p..k−1]` holds the `k−p` smallest elements of `L ∪ R`, in sorted order; and `L[i]`, `R[j]` are the smallest not-yet-copied elements of their arrays.* Since the next output must be the smaller of `L[i]` and `R[j]`, taking it preserves the invariant. On exit, one array is exhausted and the remainder of the other is already sorted and all-larger, so appending it finishes the job. [CLRS Exercise 2.3-3]

The **copy into `L` and `R`** is what makes this safe: you cannot merge two adjacent runs in place without either extra space or a much cleverer (and slower) rotation-based algorithm.

Complexity

MERGE is Θ(n) where `n = r − p + 1`. Justification, not assertion: lines 1–4 are O(1); the copies take `Θ(n_L + n_R) = Θ(n)`; and across the three `while` loops **every element is copied back into `A` exactly once**, so they total `n` iterations of constant work. [CLRS §2.3, p.38]

The recurrence.

- Divide: computing the midpoint, `D(n) = Θ(1)`.
- Conquer: `2T(n/2)`.
- Combine: `C(n) = Θ(n)`.

```
T(n) = 2T(n/2) + Θ(n)        →   T(n) = Θ(n log n)
```

Why, without invoking the master theorem — the recursion tree [CLRS Fig. 2.5, p.43]. Take `T(n) = 2T(n/2) + c₂n` with `T(1) = c₁`.

```
level 0:                    c₂n                          total  c₂n
level 1:          c₂n/2              c₂n/2               total  c₂n
level 2:    c₂n/4   c₂n/4    c₂n/4   c₂n/4               total  c₂n
   ⋮                         ⋮
level lg n:  c₁ c₁ c₁ c₁ … c₁   (n leaves)               total  c₁n
```

At depth `i` there are `2ⁱ` nodes each costing `c₂·(n/2ⁱ)`, so **every internal level costs exactly `c₂n`** — the doubling of node count and the halving of per-node cost cancel. The tree has `lg n + 1` levels (proved by induction on `n = 2ⁱ`). Total:

```
T(n) = c₂ n lg n + c₁ n = Θ(n log n)
```

	TIME	SPACE
Best / Average / Worst	Θ(n log n) — identical	Θ(n) auxiliary + Θ(log n) recursion stack

The uniformity across cases is merge sort's selling point: no input makes it slow.

C++ Implementation

```
#include <vector>
#include <utility>
```

```cpp
namespace detail {

// Merges the sorted runs v[lo..mid] and v[mid+1..hi] using scratch
as buffer.
template <typename T>
void merge(vector<T>& v, int lo, int mid, int hi, vector<T>&
scratch) {
    const int nL = mid - lo + 1;
    const int nR = hi - mid;

    // Copy both runs out; merging back into v is then safe.
    for (int i = 0; i < nL; ++i) scratch[i]       = v[lo + i];
    for (int j = 0; j < nR; ++j) scratch[nL + j]  = v[mid + 1 + j];

    int i = 0, j = 0, k = lo;
    while (i < nL && j < nR) {
        // '<=' takes from the LEFT run on ties -> stable.
        if (!(scratch[nL + j] < scratch[i])) v[k++] =
move(scratch[i++]);
        else                                 v[k++] =
move(scratch[nL + j++]);
    }
    while (i < nL) v[k++] = move(scratch[i++]);
    while (j < nR) v[k++] = move(scratch[nL + j++]);
}

template <typename T>
void mergeSortRec(vector<T>& v, int lo, int hi, vector<T>& scratch)
{
    if (lo >= hi) return;                  // 0 or 1 element
    const int mid = lo + (hi - lo) / 2;    // overflow-safe midpoint
    mergeSortRec(v, lo, mid, scratch);
    mergeSortRec(v, mid + 1, hi, scratch);
    if (v[mid + 1] < v[mid])               // cheap skip when
already ordered
        merge(v, lo, mid, hi, scratch);
}

}  // namespace detail

// Θ(n log n) worst case, Θ(n) auxiliary space, stable.
```

```
template <typename T>
void mergeSort(vector<T>& v) {
    if (v.size() < 2) return;
    vector<T> scratch(v.size());
    detail::mergeSortRec(v, 0, static_cast<int>(v.size()) - 1,
scratch);
}
```

Implementation notes

- **Allocate the scratch buffer once**, at the top. Allocating inside `merge` turns an `O(n log n)` sort into an allocation benchmark — this is the single most common performance bug in hand-written merge sorts.
- `mid = lo + (hi - lo)/2` rather than `(lo + hi)/2`: the latter overflows `int` for large arrays. Same bug that lived in Java's `binarySearch` for nine years.
- `if (v[mid+1] < v[mid])` skips the merge entirely when the two runs are already in order. Free `O(n)` best case on sorted input, one comparison of cost.
- Comparing with `!(right < left)` rather than `left <= right` means the type only needs `operator<`, matching STL conventions.
- CLRS's `MERGE` uses 1-origin indexing for `A` and 0-origin for `L`/`R` — a deliberate mixing that simplifies its loop invariant [CLRS footnote 12, p.36]. The C++ version above is uniformly 0-indexed.

Common bugs

- Merging in place without a copy. Silently wrong.
- `mid + 1` vs `mid` in the right-half recursive call → infinite recursion when `hi = lo + 1`.
- Forgetting one of the two drain loops.
- Using `<` instead of `<=` on the tie (or, as above, `right < left` instead of `left < right`) → **loses stability**.
- Reallocating scratch per call.

Recognition pattern

Merge sort is the answer when: **worst-case** $\Theta(n \log n)$ is required; **stability** is required; the data is a **linked list** (merge sort is the natural list sort, and needs no extra space there); or the data is **external / streamed** and does not fit in memory. Also the go-to when a problem says *"count something while sorting"* — counting inversions in $\Theta(n \log n)$ is merge sort with one extra line [CLRS Problem 2-4].

Alternatives

ALTERNATIVE	TRADE-OFF VS MERGE SORT
Quicksort	In-place, better constants, but $\Theta(n^2)$ worst case and unstable
Heapsort	$\Theta(n \log n)$ worst case *and* $O(1)$ space, but unstable and cache-hostile
`std::sort`	Introsort: quicksort + heapsort fallback + insertion sort base. Not stable.
`std::stable_sort`	Merge sort (adaptive; degrades to in-place merge if allocation fails)

---

11. Efficiency as a first-class technology

The argument [CLRS §1.2, p.12]

Insertion sort takes roughly $c_1 n^2$; merge sort roughly $c_2 \, n \lg n$, with $c_1 < c_2$. Write them as $c_1 \cdot n \cdot n$ and $c_2 \cdot n \cdot \lg n$: where one has a factor n, the other has $\lg n$. For $n = 1000$, $\lg n \approx 10$; for $n = 10^6$, $\lg n \approx 20$. **No matter how much smaller c_1 is, there is always a crossover beyond which merge sort wins.**

CLRS's concrete demonstration, worth carrying around because it is startling:

	COMPUTER A	COMPUTER B
Speed	10^{10} instr/sec	10^7 instr/sec (**1000× slower**)
Algorithm	insertion sort, hand-coded in machine language: $2n^2$	merge sort, average programmer, bad compiler: $50 \, n \lg n$
Sort 10^7 numbers	$2 \cdot (10^7)^2 / 10^{10}$ = **20,000 s ≈ 5.5 hours**	$50 \cdot 10^7 \cdot \lg(10^7) / 10^7$ ≈ **1163 s < 20 min**
Sort 10^8 numbers	**> 23 days**	**< 4 hours**

The slower machine with the better algorithm wins by **17×**, and the gap widens with `n`.

> ***You should consider algorithms, like computer hardware, as a technology.*** *Total system performance depends on choosing efficient algorithms as much as on choosing fast hardware.*

On machine learning (CLRS's position, worth knowing)

CLRS addresses head-on whether ML makes algorithms obsolete:

> *Machine learning is itself a collection of algorithms, just under a different name. Furthermore, it currently seems that the successes of machine learning are mainly for problems for which we, as humans, do not really understand what the right algorithm is. … For algorithmic problems that humans understand well, such as most of the problems in this book, efficient algorithms designed to solve a specific problem are typically more successful than machine-learning approaches. [CLRS §1.2, p.14]*

Estimation [Skiena §1.9, p.25]

Skiena adds a skill CLRS doesn't teach: **principled guessing**.

> *Estimation problems are best solved through some kind of logical reasoning process, typically a mix of principled calculations and analogies.*

His jar-of-pennies example (actual answer 1879) attacked three ways — volume (`((10×10)×(π×2.5²) ≈ 1962`), weight (181 pennies/lb × 10 lb ≈ 1810), analogy (2 layers × 10 rolls × 50 ≈ 1000) — all within a factor of two.

> ***A best practice in estimation is to try to solve the problem in different ways and see if the answers generally agree in magnitude.***

Why this belongs in an algorithms course: before you optimize, estimate. "Will `n²` finish?" is answered by knowing that a modern CPU does roughly 10^8–10^9 simple operations per second, so `n² ≤ 10⁸` means `n ≤ 10⁴`. That single estimate decides your algorithm in most competitive programming problems.

Outside / Engineering Context

The competitive-programming rule of thumb, not in either book but consistent with both: with a 1-second limit, budget ~10^8 elementary operations. Reading off feasible `n`:

COMPLEXITY	FEASIBLE n (1 S)
`O(n!)`	$\leqslant 11$
`O(2ⁿ · n)`	$\leqslant 20$
`O(n³)`	$\leqslant 400$
`O(n² )`	$\leqslant 10^4$
`O(n √n)`	$\leqslant 10^5$
`O(n log n)`	$\leqslant 10^6$
`O(n)`	$\leqslant 10^7$–10^8

Read it backwards: the constraint in the problem statement tells you the intended complexity.

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
Problem vs instance	A problem is a spec (legal inputs + required output properties); an instance is one input.
Algorithm vs heuristic	Algorithm = correct on **every** instance. Heuristic = usually good, no guarantee.
Correctness	Halts in finite time **and** outputs a correct answer, for all instances.
Counterexample method	Think small · think exhaustively · hunt the weakness · go for a tie · seek extremes.
Good counterexample	Verifiable + simple.
Specification traps	Input class too broad · ill-defined question · compound goals.
Narrowing instances	Restricting the input class is honorable and often the whole solution.
Loop invariant	Initialization + Maintenance + Termination. Termination is where the theorem comes from.
Induction pitfalls	Boundary errors; cavalier extension claims; forgetting to strengthen the hypothesis.
Contradiction	Assume false → derive a ridiculous consequence. Muddy outcomes don't convince.
Modeling	Map the application onto permutations / subsets / trees / graphs / points / polygons / strings.
Recursive objects	All seven decompose; that decomposition *is* the algorithm.
RAM model	One step per simple op and per memory access; loops/subroutines are not simple; bounded word size; no cache model.
Worst case default	Upper-bound guarantee; often occurs; average is often just as bad.
Insertion sort	$\Theta(n + \text{inversions})$; $\Theta(n)$ best, $\Theta(n^2)$ worst; stable, in-place, online.
Merge sort	$\Theta(n \log n)$ in all cases; $\Theta(n)$ space; stable; recurrence $T(n) = 2T(n/2) + \Theta(n)$.
Divide & conquer	Divide → Conquer → Combine, plus a base case.
Recursion tree	Every level costs $c_2 n$; $\lg n + 1$ levels; total $\Theta(n \log n)$.
Efficiency as technology	Better asymptotics on 1000× slower hardware still wins by 17× at $n = 10^7$.

Recognition Table

CLUE IN THE PROBLEM	WHAT IT POINTS TO
"Best / optimal / most relevant" with no metric	The spec is incomplete — pin the objective first
Requirement with "and also not too much X"	Compound goal → expect DP or NP-hardness
"Always pick the closest / biggest / shortest"	Greedy candidate → hunt for a counterexample (tie, extreme) before trusting it
Input can be restricted (tree instead of graph, 1-D instead of 2-D)	Narrow the instance class; a polynomial algorithm may appear
"Arrangement / tour / ordering"	Permutations
"Selection / group / committee / collection"	Subsets
"Hierarchy / ancestor / taxonomy"	Trees
"Network / circuit / relationship"	Graphs
"Sites / positions / locations"	Points
"Shapes / regions / boundaries"	Polygons
"Text / patterns / labels"	Strings
Nearly-sorted input, small `n`, online arrival	Insertion sort
Worst-case `n log n` required, or stability required, or linked list, or external	Merge sort
"Count pairs while sorting"	Merge sort with a counter (inversions)
Loop that builds a partial answer	Prove with a loop invariant
Recursion halving the input	Strong induction + recursion tree

---

Common Mistakes Recap

1. Defending a heuristic because you couldn't find a counterexample in 30 seconds.
2. Proving the invariant's initialization and maintenance, then forgetting termination — which is the only part that yields the theorem.

3. Assuming the optimal solution for `n` extends the optimal solution for `n−1` (Skiena Fig. 1.8 kills this).
4. Using an inductive hypothesis about `n−1` for a recursion that halves.
5. Modeling before validating the model on a hand-worked small instance (the lottery war story).
6. Treating `sort`, `exponentiation`, or a library call as one RAM step.
7. Quoting an average-case bound without stating the input distribution.
8. Allocating the merge scratch buffer inside the recursion.
9. `(lo + hi)/2` overflow.
10. Losing stability with a `<` / `<=` slip in the merge comparison.

Self-Test

Answer without looking. If you can't, the section is named next to the question.

1. Give the two-part definition of a problem specification, and name the three ways specifications go wrong. *(§3)*
2. Nearest-neighbour TSP: give the counterexample and explain why no starting-point rule saves it. *(§2)*
3. Why is "earliest completion first" optimal for interval scheduling, while "shortest first" and "earliest start first" are not? *(§2)*
4. State the three obligations of a loop-invariant proof. Which one produces the theorem, and how? *(§4)*
5. State the loop invariant for insertion sort and discharge termination. *(§8)*
6. Prove `Increment(y)` correct. Why must the inductive hypothesis be strengthened? *(§4)*
7. Name the seven combinatorial objects and give the recursive decomposition of each. *(§5)*
8. State three RAM-model assumptions and one thing the model deliberately ignores. *(§6)*
9. Give three reasons CLRS analyzes worst case by default. *(§7)*
10. Insertion sort's running time is `θ(n + X)`. What is `X`, and how does it explain both the best and worst case? *(§8)*
11. Derive `MERGE`'s `θ(n)` bound from its loop structure — don't just assert it. *(§10)*
12. Draw the recursion tree for `T(n) = 2T(n/2) + c₂n` and explain why every level

costs the same. *(§10)*

13. Computer A is 1000× faster than B, runs $2n^2$; B runs $50\ n \lg n$. Who wins at $n = 10^7$, and by how much? *(§11)*
14. Estimate the largest n for which an $O(n^2)$ algorithm finishes in one second, and say what you assumed. *(§11)*

Practice — where to drill this module

These are the problems that actually exercise M01's ideas. Numbers are LeetCode's.

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Insertion sort, exactly as written	147 · Insertion Sort List	forces you to write the shift-and-insert loop on a structure with no random access
`MERGE` on its own	88 · Merge Sorted Array	the merge step in isolation — and in-place from the back, which is the trick `MERGE` avoids by copying
Merge sort end to end	912 · Sort an Array · 148 · Sort List	912 wants an $O(n \lg n)$ sort you wrote yourself; 148 is merge sort on a linked list, where it is genuinely the best choice
Merging many runs	23 · Merge k Sorted Lists	the divide-and-conquer generalization of `MERGE`
Earliest-completion-first (movie scheduling)	435 · Non-overlapping Intervals · 452 · Minimum Number of Arrows to Burst Balloons · 646 · Maximum Length of Pair Chain	all three are `OptimalScheduling` with the words changed; if you can see that, the module landed
Interval bookkeeping	57 · Insert Interval	the "specify the problem precisely" discipline — the edge cases are the whole problem
Why exhaustive TSP is hopeless	847 · Shortest Path Visiting All Nodes	the $2^n \cdot n$ Held–Karp shape, which is what "exponential but not $n!$" buys you

Beyond LeetCode. CSES Problem Set — the *Sorting and Searching* section is the closest thing to a curriculum for this module. Codeforces problemset, `sortings` tag and `greedy` tag for counterexample-hunting practice under time pressure.

How to drill the actual skill of this module — which is *counterexample hunting*, not coding: take any greedy rule you invent, and before writing a line, spend two minutes trying to break it with (a) tiny instances, (b) ties, (c) extreme ratios, (d) a rotation or reflection of a "fixed" instance. Skiena's whole point is that this is a *method*.

C++ Toolkit for This Module

*Language material from Weiss, **Data Structures and Algorithm Analysis in C++**, 4th ed., ch. 1. Everything below is assumed by the appendix code.*

Every snippet in these notes compiles under `g++ -std=c++17 -Wall -Wextra` with this prelude:

```
#include <bits/stdc++.h>
using namespace std;
```

1. Parameter passing — the decision you make on every function

Weiss gives a two-part test [§1.5.3, p.27]:

> 1. *If the formal parameter should be able to change the value of the actual argument,* ***use call-by-reference*** *(`T&`).*
> 2. *Otherwise: a primitive type goes* ***by value*** *(`T`); a class type goes* ***by constant reference*** *(`const T&`), "unless it is an unusually small and easily copyable type."*

SIGNATURE	NAME	USE FOR
`void f(int n)`	call-by-value	small, unmodified
`void f(const vector<Point>& P)`	call-by-**constant**-reference	large, unmodified — **the default for containers**
`void f(vector<int>& A)`	call-by-reference	anything the function must modify
`void f(vector<int>&& A)`	call-by-rvalue-reference	you intend to *move* from a temporary

Weiss's own example of the mistake: `string randomItem(vector<string> arr)` copies the entire vector *"a tremendously expensive operation compared to the cost of computing and returning a randomly chosen array index, and… completely unnecessary."* The appendix's `nearestNeighborTour(const vector<Point>& P, int start)` shows both halves of the rule in one signature.

2. Return passing — and why returning a `vector` is now free

Weiss [§1.5.4, p.28–29]. Returning a large object used to mean a copy, which is why old C++ code passes an output parameter instead. **In C++11 and later, return-by-value moves.** `partialSum` in Weiss's Figure 1.13 returns a `vector<int>` by value and *"the vector implementation is optimized to allow this to be done with little more than a pointer change."*

So `vector<int> nearestNeighborTour(...)` is the right signature. Do **not** contort it into `void f(..., vector<int>& out)` for performance — that reflex is a decade out of date.

The exception Weiss flags: returning a reference into an existing object (`const T& randomItem2(...)`) still avoids a copy, **but only if the caller also uses a reference** — `auto& x = randomItem2(vec);`. Write `auto x = ...` and you copied anyway.

3. References as aliases, and the range-`for` trap

Weiss [§1.5.2, p.24–25]. An lvalue reference is *"another name for the object it references."* Three uses, all of which appear in this module's code:

```
void rangeForDemo(vector<int>& arr, const vector<string>& names) {
    for (auto x : arr) ++x;           // BROKEN: x is a COPY; arr is
unchanged
    for (auto& x : arr) ++x;          // works:  x is another NAME
for each element
    for (const auto& s : names)       // read-only, and no copy of
each string
        (void)s.size();               // (void) silences the unused-
value warning
}
```

The appendix writes `for (const Interval& j : I)` for exactly this reason: `Interval` holds a `string`, so a by-value loop variable would copy that string on every iteration.

4. `const` correctness

`const` on a parameter (`const vector<Point>&`) promises the function will not modify it. `const` after a member function (`double area() const`) promises the function will not modify the object. Both are checked by the compiler, and both are how a reader knows what a function does without reading it. `const int n = (int)P.size();` also documents "this never changes" to the reader *and* to the optimizer.

5. Templates — writing the algorithm once

Weiss [§1.6.1, p.37]: a function template *"is not an actual function, but instead is a pattern for what could become a function."* The sorting code in the appendix is templated on `Comparable`, which Weiss defines as a type providing `operator<` (and a copy constructor and `operator=`):

```
template <typename Comparable>
void insertionSort1Indexed(vector<Comparable>& A, int n);
```

Weiss's warning applies directly: *"it is customary to include, prior to any template, comments that explain what assumptions are made about the template argument(s)."* A template that needs `<` and is handed a type without `<` fails at **instantiation**, with an error message pointing inside your function rather than at the call — which is why the assumption goes in a comment.

6. Lambdas and function objects — passing a comparison

Weiss motivates function objects [§1.6.4, p.41] with exactly the problem `OptimalScheduling` has: an `Interval` has no natural `<`. Do we order by start, by finish, by length? The answer is *"completely decoupled from the objects"* and passed in. Weiss's mechanism is a class with `operator()`; C++11's lambda is the same thing with less typing:

```
struct Job { int start, finish; };

void sortByFinish(vector<Job>& I) {
    sort(I.begin(), I.end(),
         [](const Job& a, const Job& b) { return a.finish <
b.finish; });
}
```

That lambda **is** a function object — the compiler writes the class for you. `sort` requires a **strict weak ordering**: `cmp(a,a)` must be `false`. Writing `<=` here is undefined behaviour and really does crash `std::sort` in libstdc++ on large inputs.

7. `size()` returns an unsigned type

`vector::size()` returns `size_t`. `for (int i = 0; i < (int)v.size() - 1; ++i)` is safe; `for (int i = 0; i < v.size() - 1; ++i)` on an **empty** vector computes `0u - 1 = 18446744073709551615` and loops forever. Every appendix function casts once: `const int n = (int)P.size();`.

Appendix — C++ for Every Pseudocode Block

Every pseudocode block in this module, translated **literally**, line for line, with the pseudocode's own variable names kept wherever C++ allows. These are deliberately *not* the tuned versions — the point is that you can put the pseudocode and the C++ side by side and see the correspondence.

All of it compiles as one translation unit under `g++ -std=c++17 -Wall -Wextra` and is exercised by the tests quoted at the end of each entry.

A1 NearestNeighbor

Pseudocode: §2, "Case study 1: Robot tour optimization".

```
// A point in the plane. Aggregate initialisation ({1.0, 2.0})
works because
// this is a plain struct with public members and no user-declared
constructor.
// The `= 0` are default member initialisers (C++11): Point p;
gives {0,0}
// rather than garbage, which is a real class of bug removed for
free.
struct Point {
    double x = 0, y = 0;
};

// `const Point&` — call-by-constant-reference [Weiss §1.5.3,
p.26]. A Point is
```

```cpp
// only 16 bytes, so by-value would be fine here; the reference form is written
// out because it is the habit you want for every larger type.
double dist(const Point& a, const Point& b) {
    return hypot(a.x - b.x, a.y - b.y);   // hypot avoids overflow that sqrt(dx*dx+dy*dy) can hit
}

// Returns the visiting ORDER as indices into P, not the points themselves:
// indices are cheap to copy and let the caller map back to whatever it likes.
// Return-by-value is correct and free — C++11 moves the vector out [Weiss §1.5.4].
vector<int> nearestNeighborTour(const vector<Point>& P, int start = 0) {
    const int n = (int)P.size();          // cast ONCE: size() is unsigned (size_t)
    if (n == 0) return {};                // `{}` value-initialises the return type: an empty vector

    // vector<char>, not vector<bool>! vector<bool> is a bit-packed SPECIALISATION
    // that does not behave like a normal container (operator[] returns a proxy,
    // you cannot take &v[i]). For flags, vector<char> is the safe default.
    vector<char> visited(n, 0);

    vector<int> tour;
    tour.reserve(n);                      // one allocation instead of O(lg n) reallocations

    int p = start;                        // "Pick and visit an initial point p0 from P"
    visited[p] = 1;
    tour.push_back(p);

    for (int step = 1; step < n; ++step) {          // "While there are still unvisited points"
        int best = -1;
```

```
        double bestD = numeric_limits<double>::infinity();   //
<limits>: the honest "infinity"
        for (int q = 0; q < n; ++q) {                // "Select p_i
closest to p_{i-1}"
            if (visited[q]) continue;
            double d = dist(P[p], P[q]);
            if (d < bestD) { bestD = d; best = q; }
        }
        visited[best] = 1;
        tour.push_back(best);
        p = best;
    }
    return tour;                             // "Return to p0" is
implicit: the tour is a cycle
}

// Cost of a closed tour. The modulo is what closes the cycle: the
last point's
// successor is tour[0].
double tourLength(const vector<Point>& P, const vector<int>& tour)
{
    double total = 0;
    for (size_t i = 0; i < tour.size(); ++i)
        total += dist(P[tour[i]], P[tour[(i + 1) % tour.size()]]);
    return total;
}
```

Complexity. `Θ(n²)`: `n - 1` rounds, each scanning all `n` points. **Correct? No — this is a heuristic.**

> *Verified: on Skiena's line instance (points at `-21, -5, -1, 0, 1, 3, 11`, starting at `0`) the code visits `0, -1, 1, 3, -5, -21, 11` — the zigzag across the origin, exactly as Figure 1.3 shows. Tour length **72** against the **exact optimum 64** (computed by Held–Karp over all orderings): **12.5% worse.***

A2 ClosestPair

Pseudocode: §2, "Attempt 2 — Closest pair".

```
// The pseudocode says "endpoints from distinct vertex chains", so
we need to
```

```
// know which chain a point belongs to. That is a union-find (M10)
— here in its
// smallest useful form, with path halving and no union-by-rank.
struct ChainSet {
    vector<int> parent;
    // `explicit` stops the compiler from silently converting an
int into a
    // ChainSet [Weiss §1.4.2]. Single-argument constructors should
almost
    // always be explicit.
    explicit ChainSet(int n) : parent(n) {        // member-
initialiser list, not assignment
        for (int i = 0; i < n; ++i) parent[i] = i;
    }
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x =
parent[x]; }
        return x;
    }
    void join(int a, int b) { parent[find(a)] = find(b); }
};

// Returns the tour as a list of edges. pair<int,int> is the
lightweight
// two-field struct the standard library already wrote for us.
vector<pair<int,int>> closestPairTour(const vector<Point>& P) {
    const int n = (int)P.size();
    if (n < 2) return {};
    vector<pair<int,int>> edges;
    vector<int> deg(n, 0);                // a tour is a cycle:
every vertex ends with degree 2
    ChainSet chain(n);

    for (int i = 1; i <= n - 1; ++i) {                // "For i =
1 to n-1"
        double d = numeric_limits<double>::infinity(); // "d =
infinity"
        int sm = -1, tm = -1;
        for (int s = 0; s < n; ++s) {                 // "For each
pair of endpoints (s,t)"
            if (deg[s] >= 2) continue;                //   an
endpoint has degree < 2
```

```
            for (int t = s + 1; t < n; ++t) {
                if (deg[t] >= 2) continue;
                // "from distinct vertex chains" — this is the
premature-cycle guard.
                // On the very last edge we WANT to close the
cycle, so we drop it.
                if (i < n - 1 && chain.find(s) == chain.find(t))
continue;
                double dst = dist(P[s], P[t]);
                if (dst <= d) { d = dst; sm = s; tm = t; }   //
`<=` matches the pseudocode
            }
        }
        edges.push_back({sm, tm});                      // "Connect
(s_m, t_m) by an edge"
        ++deg[sm]; ++deg[tm];
        chain.join(sm, tm);
    }

    // "Connect the two remaining endpoints by an edge."
    int a = -1, b = -1;
    for (int v = 0; v < n; ++v) if (deg[v] < 2) { (a < 0 ? a : b) =
v; }
    // The conditional operator yields an LVALUE when both arms are
lvalues of
    // the same type, so it can appear on the LEFT of `=`. Legal,
and a compact
    // way to say "fill a first, then b".
    if (a >= 0 && b >= 0) edges.push_back({a, b});
    return edges;
}
```

Complexity. `Θ(n³)` as written: `n − 1` rounds × `O(n²)` endpoint pairs. **Also a heuristic.**

> *Verified: on the two-row instance (rows `1 − ε` apart, neighbours within a row `1 + ε` apart, `ε = 0.01`) against the **exact** optimum from Held–Karp:*

> | *POINTS* | *CLOSEST-PAIR HEURISTIC* | *EXACT OPTIMUM* | *WORSE BY* |
> |---|---|---|---|
> | *8* | *9.2196* | *8.0400* | *14.7%* |
> | *12* | *16.5404* | *12.0800* | *36.9%* |
> | *16* | *22.5333* | *16.1200* | ***39.8%*** |
>
> *Skiena's figure claims "over 20%"; with more points it is worse than that, and the gap keeps growing. Every returned tour was a genuine cycle — every vertex had degree exactly 2.*

A3 OptimalScheduling

Pseudocode: §2, "Case study 2: Movie scheduling".

```
struct Interval {
    int start = 0, finish = 0;
    string name;                        // a std::string member is why
we pass Interval by const&
};

// Takes its argument BY VALUE on purpose: we need to sort, and
sorting the
// caller's vector would be a surprise. Taking by value + sorting
the local copy
// is the honest signature, and if the caller passes a temporary
// (`optimalScheduling(makeIntervals())`) the copy is elided into a
move.
vector<Interval> optimalScheduling(vector<Interval> I) {
    // "the job with the earliest completion date" — so order by
`finish`.
    // The lambda IS a function object [Weiss §1.6.4, p.42]; the
compiler
    // generates a class with operator() from it.
    // Must be a STRICT weak ordering: `<`, never `<=`.
    sort(I.begin(), I.end(),
         [](const Interval& a, const Interval& b) { return a.finish
< b.finish; });

    vector<Interval> accepted;
    int lastFinish = numeric_limits<int>::min();       // "nothing
accepted yet"
```

```
    // `const Interval&` in the range-for: no copy of the string
member per
    // iteration [Weiss §1.5.2, p.25].
    for (const Interval& j : I) {
        // "Delete j, and any interval intersecting j, from I" —
sorting by
        // finish time turns that deletion into a single
comparison: anything
        // starting before lastFinish intersects something already
accepted.
        if (j.start >= lastFinish) {
            accepted.push_back(j);
            lastFinish = j.finish;
        }
    }
    return accepted;
}
```

Complexity. $\Theta(n \lg n)$ for the sort, $\Theta(n)$ for the sweep. The pseudocode's literal "delete every intersecting interval" would be $\Theta(n^2)$; sorting by finish time is what collapses it.

This one is not a heuristic — it is optimal, and the proof is the exchange argument sketched in §2 and formalized in M12.

> *Verified: on the War and Peace instance it picks 4 of 5. Against **brute force over all 2^n subsets** on 500 random instances ($n \le 10$), the greedy answer matched the true optimum **every time**.*

A4 Increment

Pseudocode: §4, "Strengthening the hypothesis".

```
// Skiena's deliberately odd function. The point is not the code —
it is that
// proving it correct REQUIRES strong induction, because the
recursion drops
// from y to floor(y/2), not to y-1.
//
// `long long` because the doubling on the odd branch makes
intermediate values
```

```
// grow if you get the recursion wrong, and because 32-bit overflow
is UB.
long long incrementSkiena(long long y) {
    if (y == 0) return 1;                                  // if (y =
0) return 1
    if (y % 2 == 1) return 2 * incrementSkiena(y / 2); // odd: 2 *
Increment(floor(y/2))
    return y + 1;                                          // even: y +
1
}
// NOTE on integer division: for NON-NEGATIVE y, `y / 2` in C++ is
exactly
// floor(y/2), which is what the pseudocode means. For NEGATIVE
operands C++
// truncates toward zero, so -3 / 2 == -1, NOT floor(-1.5) == -2.
Any time
// pseudocode says floor() and your input can be negative, that
mismatch is a
// real bug — here the function is only defined for y >= 0.
```

Why it works. For odd `y = 2m + 1`: `2·Increment(m) = 2(m + 1) = 2m + 2 = y + 1`. ✓ The inductive hypothesis has to be *"holds for all `k ≤ y − 1`"*, not just `y − 1`, because the call is to `m ≈ y/2`.

> *Verified:* `incrementSkiena(y) == y + 1` *for every* `y` *in* `[0, 200000)`.

A5 INSERTION-SORT

Pseudocode: §8, "Pseudocode (CLRS, 1-indexed)".

```
// CLRS indexes A[1..n]. Rather than silently shifting to 0-based
(which is what
// the practical version in the body does), this translation keeps
1-based
// indexing so every line matches the book: the caller passes a
vector of size
// n+1 whose slot 0 is unused. Wasting one element to avoid off-by-
one bugs in a
// transcription is a fair trade.
//
```

```
// TEMPLATE ASSUMPTION [Weiss §1.6.1, p.37]: Comparable must
provide operator>
// (used at the loop test), a copy constructor, and operator=. int,
double and
// string all qualify.
template <typename Comparable>
void insertionSort1Indexed(vector<Comparable>& A, int n) {   //
`&`: we sort in place
    for (int i = 2; i <= n; ++i) {          // 1  for i = 2 to n
        Comparable key = A[i];              // 2      key = A[i]
   (a real COPY — needed:
                                            //        A[i] is about
to be overwritten)
        int j = i - 1;                      // 4      j = i - 1
        // 5  while j > 0 and A[j] > key
        // The order of the && operands matters: C++ SHORT-
CIRCUITS, so `j > 0`
        // is tested first and A[j] is never read at j == 0. Swap
them and you
        // read A[0]... which here exists, so the bug would be
silent. In the
        // 0-indexed version it would be out-of-bounds.
        while (j > 0 && A[j] > key) {
            A[j + 1] = A[j];                // 6          A[j+1] =
A[j]
            j = j - 1;                      // 7          j = j - 1
        }
        A[j + 1] = key;                     // 8      A[j+1] = key
    }
}
```

Complexity. $\Theta(n)$ best case (already sorted — the `while` never fires), $\Theta(n^2)$ worst and average. Space $\Theta(1)$ auxiliary: it sorts **in place**.

Stable, because `A[j] > key` is strict: an element *equal* to `key` stops the shift, so equal elements keep their original relative order.

> *Verified: agrees with `std::sort` on 400 random arrays of length 0–59.*

A6 MERGE-SORT and MERGE

Pseudocode: §10, "Pseudocode".

```
// MERGE(A, p, q, r): A[p..q] and A[q+1..r] are each sorted; make
A[p..r] sorted.
template <typename Comparable>
void merge1Indexed(vector<Comparable>& A, int p, int q, int r) {
    const int nL = q - p + 1;               // 1  n_L = q - p + 1
    const int nR = r - q;                   //    n_R = r - q

    // 2-3  "let L and R be new arrays; copy A[p..q] into L,
A[q+1..r] into R"
    // The iterator-pair constructor copies a RANGE. Note the
asymmetry that
    // trips everyone up: the range is [first, last) — last is ONE
PAST the end.
    // A[p..q] inclusive therefore becomes [begin+p, begin+q+1).
    vector<Comparable> L(A.begin() + p, A.begin() + q + 1);
    vector<Comparable> R(A.begin() + q + 1, A.begin() + r + 1);

    int i = 0, j = 0, k = p;                // 4  i = 0; j = 0; k =
p
    while (i < nL && j < nR) {              // 5  while i < n_L and
j < n_R
        // CLRS writes `if L[i] <= R[j]`. Comparable is only
promised operator<
        // [Weiss §1.6.3, p.39], so express "L[i] <= R[j]" as "not
(R[j] < L[i])".
        // This is the standard library's own convention and it is
what keeps
        // the merge STABLE: on a tie we take from L, the earlier
half.
        if (!(R[j] < L[i])) { A[k] = L[i]; ++i; }   // 6-7
        else                { A[k] = R[j]; ++j; }   // 8-9
        ++k;                                        // 10
    }
    while (i < nL) { A[k] = L[i]; ++i; ++k; }       // 11  drain L
    while (j < nR) { A[k] = R[j]; ++j; ++k; }       // 12  drain R
}

template <typename Comparable>
void mergeSort1Indexed(vector<Comparable>& A, int p, int r) {
```

```
    if (p >= r) return;                     // 1-2  zero or one
element: already sorted
    // 3  q = floor((p+r)/2), written to avoid overflow.
    // `(p + r) / 2` can overflow int when p and r are both large;
`p + (r-p)/2`
    // cannot, and equals the same floor for p <= r. This is the
famous bug that
    // sat in java.util.Arrays.binarySearch for nine years.
    int q = p + (r - p) / 2;
    mergeSort1Indexed(A, p, q);             // 4
    mergeSort1Indexed(A, q + 1, r);         // 5
    merge1Indexed(A, p, q, r);              // 6
}
```

Complexity. `T(n) = 2T(n/2) + Θ(n) = Θ(n lg n)`, every case — there is no best case, because the recursion does not look at the data. **Space `Θ(n)` auxiliary** for `L` and `R`, plus `Θ(lg n)` recursion stack.

The copy into `L` and `R` is not laziness. You cannot merge two adjacent sorted runs in place without either extra space or a much slower rotation-based algorithm; this is exactly why merge sort loses to quicksort in practice despite the better worst case (M05).

`Comparable` here needs only `operator<` — see the `!(R[j] < L[i])` comment. That is the same contract `std::sort` imposes, and adopting it means your code works with any type that already works with the standard library.

> *Verified: agrees with `std::sort` on 400 random arrays; and on an array of `(key, id)` pairs sorted by key alone, every run of equal keys came out with **increasing `id`** — i.e. `MERGE` as written really is stable.*

Next: M02 — Asymptotics & the Analysis Toolkit